

Thuật toán cây khung nhỏ nhất và ứng dụng

Wu Jingyue

Nguồn gốc: Minimum spanning tree algorithms and their applications

I0I China candidate team papers / 2004 / Wu Jingyue.pdf

Tóm tắt

Cây khung nhỏ nhất là một bài toán kinh điển và là một phần quan trọng của lý thuyết đồ thị. Sách giáo khoa thường chỉ giới thiệu thuật toán tìm cây khung nhỏ nhất, trong khi phần ứng dụng của thuật toán lại đặc sắc hơn nhiều. Bài viết này phân tích tương đối đầy đủ thuật toán cây khung nhỏ nhất và các ứng dụng của nó, nhằm giúp người đọc có một nhận thức sâu hơn. Nội dung gồm ba phần: phần cơ sở giới thiệu khái niệm, thuật toán tổng quát và các thuật toán thường dùng; phần ứng dụng phân tích từng bài toán cụ thể, nhấn mạnh quá trình suy nghĩ và chứng minh; cuối cùng là phần kết luận.

Từ khóa. Cây khung nhỏ nhất.

1 Cơ sở

1.1 Định nghĩa

Trong thiết kế mạch điện, ta thường cần dùng dây nối các chân của một số linh kiện điện tử. Nếu mỗi dây nối hai chân, để nối tất cả n chân lại với nhau thì chỉ cần $n - 1$ dây. Trong mọi cách nối, ta thường quan tâm đến cách có tổng chiều dài dây nhỏ nhất.

Mô hình đồ thị của bài toán như sau. Cho một đồ thị vô hướng liên thông $G = (V, E)$, trong đó V là tập các chân linh kiện, còn E là tập mọi cách nối có thể giữa hai chân. Mỗi cạnh (u, v) có trọng số $w(u, v)$, biểu diễn chiều dài dây cần dùng để nối hai chân đó. Mục tiêu là tìm một tập cạnh không chu trình T , nối được mọi đỉnh và có tổng trọng số nhỏ nhất:

$$w(T) = \sum_{(u,v) \in T} w(u, v).$$

Vì T nối mọi đỉnh và không có chu trình, nó là một cây. Cây này được sinh ra từ đồ thị G , nên được gọi là **cây khung**. Nếu một cây khung có tổng trọng số nhỏ nhất trong tất cả các cây khung, ta gọi nó là **cây khung nhỏ nhất**.

1.2 Thuật toán tổng quát

Có một phương pháp tổng quát để giải bài toán cây khung nhỏ nhất: dùng tham lam. Thuật toán duy trì một tập cạnh A luôn là tập con của một cây khung nhỏ nhất nào đó. Ở mỗi bước, ta quyết định có thêm cạnh (u, v) vào A hay không. Điều kiện được phép thêm là $A \cup \{(u, v)\}$ vẫn là tập con của một cây khung nhỏ nhất. Một cạnh như vậy được gọi là **cạnh an toàn** đối với A .

GENERIC-MST(G, w)

1. $A \leftarrow$ empty set
2. while A has not formed a spanning tree
3. find a safe edge (u, v) for A
4. $A \leftarrow A \cup \{(u, v)\}$
5. return A

Ban đầu $A = \emptyset$, hiển nhiên thỏa điều kiện là tập con của một cây khung nhỏ nhất. Sau mỗi vòng lặp, ta thêm một cạnh an toàn, nên bất biến đó vẫn được giữ. Phần còn lại của mục này đưa ra một quy tắc nhận biết cạnh an toàn; hai thuật toán Kruskal và Prim ở mục sau đều dựa trên quy tắc này.

Trước hết cần vài khái niệm. Với đồ thị vô hướng $G = (V, E)$, một **lát cắt** $(S, V - S)$ là một phân hoạch của V . Nếu một cạnh $(u, v) \in E$ có một đầu mút thuộc S và đầu mút còn lại thuộc $V - S$, ta nói cạnh đó **đi qua** lát cắt. Nếu không có cạnh nào của A đi qua lát cắt, ta nói lát cắt đó **không cản trở** A . Một cạnh đi qua lát cắt và có trọng số nhỏ nhất trong các cạnh đi qua lát cắt được gọi là **cạnh nhẹ** của lát cắt.

Định lý 1. Cho $G = (V, E)$ là đồ thị vô hướng liên thông, với hàm trọng số thực w trên E . Giả sử $A \subseteq E$ và A nằm trong một cây khung nhỏ nhất nào đó của G . Nếu $(S, V - S)$ là một lát cắt bất kỳ không cản trở A , và (u, v) là một cạnh nhẹ đi qua lát cắt đó, thì (u, v) là cạnh an toàn đối với A .

Chứng minh. Gọi T là một cây khung nhỏ nhất chứa A . Nếu T đã chứa cạnh nhẹ (u, v) , thì T cũng chứa $A \cup \{(u, v)\}$ và ta xong.

Ngược lại, xét đường đi duy nhất P từ u đến v trong T . Thêm (u, v) vào T tạo ra một chu trình. Vì (u, v) đi qua lát cắt $(S, V - S)$, trên đường đi P phải có ít nhất một cạnh cũng đi qua lát cắt này; gọi cạnh đó là (x, y) . Lát cắt không cản trở A , nên $(x, y) \notin A$.

Xóa (x, y) khỏi T sẽ tách T thành hai thành phần. Thêm (u, v) vào sẽ nối hai thành phần đó lại, tạo thành một cây khung mới

$$T' = (T - \{(x, y)\}) \cup \{(u, v)\}.$$

Do (u, v) là cạnh nhẹ qua lát cắt, còn (x, y) cũng đi qua lát cắt, ta có

$$w(u, v) \leq w(x, y).$$

Vì vậy

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T).$$

Mà T là cây khung nhỏ nhất, nên T' cũng là cây khung nhỏ nhất. Hơn nữa T' chứa $A \cup \{(u, v)\}$, do đó (u, v) an toàn đối với A .

Định lý này cũng giải thích luồng chạy của GENERIC-MST. Trong suốt quá trình, A luôn không có chu trình; nếu không, không cây khung nào có thể chứa A . Tại mọi thời điểm, $G_A = (V, A)$ là một rừng và mỗi thành phần liên thông của G_A là một cây. Mọi cạnh an toàn đối với A đều nối hai thành phần khác nhau của rừng, vì $A \cup \{(u, v)\}$ không được tạo chu trình.

Mỗi lần thêm một cạnh, số cây trong rừng giảm đi một. Ban đầu có $|V|$ cây đơn đỉnh; khi chỉ còn một cây, ta đã có $|V| - 1$ cạnh và thuật toán kết thúc.

Hệ quả 1. Cho $G = (V, E)$ là đồ thị vô hướng liên thông có trọng số w , và $A \subseteq E$ nằm trong một cây khung nhỏ nhất nào đó. Gọi C là một thành phần liên thông của rừng $G_A = (V, A)$. Nếu (u, v) là cạnh nhẹ nối C với một thành phần khác của G_A , thì (u, v) an toàn đối với A .

Chứng minh. Lát cắt $(C, V - C)$ không cản trở A . Cạnh (u, v) là cạnh nhẹ qua lát cắt này, nên theo Định lý 1, nó an toàn đối với A .

1.3 Các thuật toán thường dùng

Thuật toán Kruskal

Kruskal là một cài đặt trực tiếp của thuật toán tổng quát. Thuật toán chọn cạnh có trọng số nhỏ nhất trong số các cạnh nối hai cây bất kỳ của rừng hiện tại, rồi thêm cạnh đó vào rừng. Nếu C_1 và C_2 là hai cây được cạnh (u, v) nối lại, thì (u, v) là cạnh nhẹ nối C_1 với một thành phần khác; do Hệ quả 1, nó là cạnh an toàn. Đây là một thuật toán tham lam vì ở mỗi bước nó chọn cạnh rẻ nhất có thể.

Cài đặt Kruskal thường dùng cấu trúc hợp nhất - tìm kiếm. Mỗi tập biểu diễn một cây hiện có trong rừng. FIND-SET(u) trả về đại diện của tập chứa u , nhờ đó ta kiểm tra được hai đỉnh có nằm trong cùng một cây hay không. UNION dùng để gộp hai cây.

MST-KRUSKAL(G, w)

1. $A \leftarrow$ empty set
2. for each vertex v in $V[G]$
3. MAKE-SET(v)
4. sort edges of E by nondecreasing weight
5. for each edge (u, v) in that order
6. if FIND-SET(u) $\neq$ FIND-SET(v)
7. $A \leftarrow A \cup \{(u, v)\}$
8. UNION(u, v)
9. return A

Ban đầu A rỗng và có $|V|$ cây đơn đỉnh. Sau khi sắp xếp cạnh theo trọng số tăng dần, thuật toán xét từng cạnh (u, v) . Nếu u và v đã cùng một cây, thêm cạnh sẽ tạo chu trình nên cạnh bị bỏ qua. Nếu chúng thuộc hai cây khác nhau, cạnh được thêm vào A và hai tập được gộp lại.

Thời gian chạy phụ thuộc vào cách cài đặt cấu trúc hợp nhất - tìm kiếm. Với nén đường đi, phần khởi tạo tốn $O(V)$, sắp xếp cạnh tốn $O(E \lg E)$, còn các thao tác tập rời nhau tốn $O(E\alpha(E, V))$, trong đó α là nghịch đảo của hàm Ackermann. Vì $\alpha(E, V) = O(\lg E)$, tổng thời gian của Kruskal là

$$O(E \lg E).$$

Thuật toán Prim

Prim cũng là một trường hợp riêng của thuật toán tổng quát, và cách chạy rất giống thuật toán Dijkstra tìm đường đi ngắn nhất. Khác với Kruskal, tập cạnh A của Prim luôn tạo thành đúng một cây. Ở mỗi bước, các đỉnh trong cây và các đỉnh ngoài cây xác định một lát cắt; cạnh nhẹ đi qua lát cắt đó được thêm vào cây. Cây bắt đầu từ một đỉnh gốc tùy ý r , rồi lớn dần cho đến khi phủ hết V .

Trong giả mã dưới đây, mọi đỉnh chưa thuộc cây nằm trong hàng đợi ưu tiên Q , khóa ưu tiên là key. Với mỗi đỉnh v , key[v] là trọng số nhỏ nhất của một cạnh nối v với cây hiện tại; nếu chưa có cạnh như vậy thì key[v] là vô cùng. Mảng parent[v] lưu cha của v trong cây.

MST-PRIM(G, w, r)

1. $Q \leftarrow V[G]$
2. for each u in Q

```

3.   key[u] <- infinity
4. key[r] <- 0
5. parent[r] <- NIL
6. while Q is not empty
7.   u <- EXTRACT-MIN(Q)
8.   for each v in Adj[u]
9.     if v in Q and w(u, v) < key[v]
10.      parent[v] <- u
11.      key[v] <- w(u, v)

```

Trong quá trình chạy, tập cạnh của thuật toán tổng quát được biểu diễn ngầm bởi

$$A = \{(v, \text{parent}[v]) \mid v \in V - \{r\} - Q\}.$$

Khi thuật toán kết thúc, Q rỗng và cây khung nhỏ nhất là

$$A = \{(v, \text{parent}[v]) \mid v \in V - \{r\}\}.$$

Nếu dùng heap nhị phân để cài đặt Q , khởi tạo tốn $O(V)$. Có $|V|$ lần EXTRACT-MIN, mỗi lần $O(\lg V)$, tổng cộng $O(V \lg V)$. Vòng lặp qua danh sách kề thực hiện tổng cộng $O(E)$ lần; mỗi lần giảm khóa heap tốn $O(\lg V)$. Vì vậy tổng thời gian là

$$O(V \lg V + E \lg V) = O(E \lg V).$$

Mức này cùng bậc với Kruskal trong cách cài đặt thông thường.

1.4 Ví dụ 1: Maintain (IOI 2003)

Mô tả bài toán

Trong một đồ thị vô hướng ban đầu rỗng, các cạnh mới liên tục được thêm vào. Nếu đồ thị hiện tại liên thông, cần in tổng trọng số cây khung nhỏ nhất của đồ thị hiện tại; nếu chưa liên thông, in -1.

Phân tích

Theo kích thước dữ liệu, sau mỗi lần thêm cạnh ta không thể tính lại cây khung nhỏ nhất từ đầu. Vấn đề chính là làm thế nào thêm một cạnh vào cây khung nhỏ nhất hiện có và nhanh chóng nhận được cây khung nhỏ nhất mới.

Cây khung của đồ thị n đỉnh có đúng $n - 1$ cạnh và liên thông. Nếu thêm một cạnh mới vào cây, chắc chắn xuất hiện một chu trình. Xóa một cạnh trên chu trình đó, kể cả chính cạnh mới, ta lại thu được một cây khung. Để cây thu được là nhỏ nhất, cạnh bị xóa phải là cạnh dài nhất trên chu trình.

Thuật toán:

1. Khởi tạo đồ thị rỗng.
2. Đọc và thêm cạnh mới cho đến khi đồ thị liên thông hoặc hết input.
3. Nếu vẫn chưa liên thông, kết thúc.
4. Tính cây khung nhỏ nhất của đồ thị hiện tại.
5. Với mỗi cạnh mới (x, y) có trọng số c , tìm cạnh trọng số lớn nhất trên đường đi từ x đến y trong cây hiện tại. Nếu cạnh đó có trọng số lớn hơn c , thêm cạnh mới và xóa cạnh lớn nhất đó.

6. In tổng trọng số cây hiện tại sau mỗi lần cập nhật.

Một khung cài đặt cho bước cập nhật là:

```
update_tree(edge x-y with cost c):  
    e <- maximum-weight edge on the path x ... y in the current tree  
    if weight(e) > c:  
        remove e from the tree  
        add (x, y, c) to the tree
```

Tiểu kết

Độ khó của bài này không lớn, nhưng nó chứa những tính chất cơ bản của cây khung nhỏ nhất: thêm một cạnh vào cây khung tạo ra một chu trình; xóa cạnh lớn nhất trên chu trình đó sẽ cho một cây khung nhỏ nhất mới.

2 Ứng dụng

Giống như các thuật toán khác, để thật sự nắm được cây khung nhỏ nhất, ta không chỉ cần hiểu bản thân thuật toán mà còn phải biết dùng nó đúng lúc. Các ví dụ sau minh họa cách nhận ra mô hình cây khung nhỏ nhất trong những bài toán không mang hình thức trực tiếp của bài MST.

2.1 Ví dụ 2: Robot (BOI 2002)

Mô tả bài toán

Trong tương lai gần, robot sẽ mang đồ ăn nhanh tới các thí sinh tại Balkan Olympiad in Informatics. Đồ ăn được đặt trên một khay hình vuông. Đường từ bếp đến phòng nghỉ của thí sinh có nhiều vật cản, nên robot không thể mang khay có kích thước tùy ý. Cần xác định cạnh lớn nhất của khay vuông có thể đi qua.

Lộ trình nằm trong một hành lang tạo bởi hai bức tường song song từng đoạn, và hành lang chỉ rẽ góc 90° . Hành lang bắt đầu theo chiều dương trục x . Vật cản là các cột, được xem như các điểm nằm giữa hai bức tường. Để robot đi qua, khay không được chạm vào cột hoặc tường; chỉ mép khay được phép vừa khít với chúng. Robot và khay chỉ được tịnh tiến theo phương x hoặc phương y . Kích thước robot nhỏ hơn kích thước khay và robot luôn nằm hoàn toàn dưới khay.

Input gồm số đoạn m của mỗi bức tường, tọa độ $m + 1$ điểm gãy của bức tường trên, tọa độ $m + 1$ điểm gãy của bức tường dưới, số vật cản n và tọa độ của các vật cản. Ràng buộc là $1 \leq m \leq 30$, $0 \leq n \leq 100$, mọi tọa độ có giá trị tuyệt đối nhỏ hơn 32001. Output là cạnh lớn nhất của khay vuông.

Phân tích

Cách dễ nghĩ là nhị phân đáp án rồi mô phỏng lan truyền vùng đi được. Ý tưởng đó đơn giản, nhưng lập trình khá rườm rà. Ta thử nhìn bài toán từ một góc khác. Robot là hình vuông còn vật cản là điểm. Nếu chuyển kích thước của robot sang vật cản, robot trở thành một điểm, còn mỗi vật cản trở thành một hình vuông. Hai mô hình này tương đương.

Một điểm không đi qua được hành lang khi và chỉ khi các vật cản tạo thành một đường chắn nối hai bức tường. Vì vậy bài toán được chuyển thành đồ thị: xem mỗi vật cản và mỗi bức tường là một đỉnh. Trọng số giữa hai đỉnh được định nghĩa như sau.

- Giữa hai vật cản $p_1(x_1, y_1)$ và $p_2(x_2, y_2)$, khoảng cách là

$$\max\{|x_1 - x_2|, |y_1 - y_2|\}.$$

- Giữa một vật cản và một bức tường, khoảng cách là khoảng cách nhỏ nhất từ điểm vật cản đến mọi điểm trên bức tường theo cùng chuẩn ℓ_∞ .
- Giữa hai bức tường, khoảng cách là bề rộng nhỏ nhất của hành lang.

Xem hai bức tường là đỉnh xuất phát và đỉnh kết thúc. Khi cạnh có độ dài đủ lớn để nối các chướng ngại trên một đường đi từ tường này sang tường kia, hành lang bị chặn. Do đó bài toán trở thành: trong tất cả các đường đi từ đỉnh xuất phát đến đỉnh kết thúc, hãy tìm đường đi có cạnh lớn nhất nhỏ nhất.

Bài toán này có thể giải bằng nhị phân cộng BFS với độ phức tạp $O(n^2 \log W)$, trong đó W là phạm vi trọng số cạnh. Nhưng có cách tốt hơn: tính cây khung nhỏ nhất của đồ thị đầy đủ trên các vật cản và hai bức tường. Đường đi giữa hai bức tường trong cây khung nhỏ nhất chính là đường đi cần tìm; cạnh lớn nhất trên đường đó là đáp án. Với n vật cản, cách làm này có thể cài đặt trong $O(n^2)$.

Chứng minh. Gọi hai đỉnh tường là u và v . Trong cây khung nhỏ nhất, xét đường đi duy nhất từ u đến v , gọi là đường cũ, và gọi cạnh lớn nhất trên đường này là m . Giả sử tồn tại một đường khác từ u đến v mà mọi cạnh đều ngắn hơn m .

Nếu đường mới chứa m , cạnh lớn nhất của nó không thể ngắn hơn m , mâu thuẫn. Nếu đường mới không chứa m , nó phải đi vòng qua cạnh m . Xóa m khỏi cây khung nhỏ nhất, cây bị tách thành hai cây con. Trên đường mới từ u đến v phải có một cạnh m' nối hai cây con này. Cạnh m' không thuộc cây khung nhỏ nhất; nếu thuộc, sau khi xóa m hai đầu của nó vẫn cùng thành phần, trái với việc nó nối hai cây con.

Vì mọi cạnh trên đường mới đều ngắn hơn m , ta có $w(m') < w(m)$. Thay m bằng m' sẽ tạo ra một cây khung có tổng trọng số nhỏ hơn cây khung nhỏ nhất, mâu thuẫn. Vậy không tồn tại đường nào từ u đến v có cạnh lớn nhất nhỏ hơn m . Đường trên cây khung nhỏ nhất là đường tối ưu.

Tiểu kết

Bài này có nhiều lời giải: nhị phân và mô phỏng, nhị phân và BFS, hoặc một biến thể Dijkstra cho tiêu chí minimax. Dùng cây khung nhỏ nhất không chỉ hiệu quả và gọn hơn, mà còn gợi mở hơn. Một bài toán thoát nhìn thuộc hình học tính toán được chuyển sang mô hình đồ thị; sau đó bài toán đồ thị lại được giải bằng tính chất minimax của cây khung nhỏ nhất. Điều này cho thấy ứng dụng của MST đòi hỏi khả năng quan sát, nền tảng đồ thị chắc và lập luận chặt chẽ.

2.2 Ví dụ 3: Arctic Network (Waterloo University 2002)

Mô tả bài toán

Một vùng Bắc Cực có n làng, $1 \leq n \leq 500$. Tọa độ của mỗi làng là một cặp số nguyên (x, y) với $0 \leq x, y \leq 10000$. Để tăng cường liên lạc, cần xây dựng một mạng truyền thông giữa các làng. Công cụ liên lạc có thể là bộ thu phát vô tuyến hoặc thiết bị vệ tinh. Mọi làng đều có thể được cấp một bộ thu phát vô tuyến cùng kiểu; còn số thiết bị vệ tinh bị giới hạn và chỉ cấp được cho một số làng.

Một kiểu thu phát vô tuyến có tham số d : hai làng có khoảng cách không vượt quá d thì liên lạc trực tiếp bằng vô tuyến được. d càng lớn, thiết bị càng đắt. Hai làng đều có thiết bị vệ tinh thì liên lạc trực tiếp được bất kể khoảng cách.

Cho k thiết bị vệ tinh, $0 \leq k \leq 100$. Cần phân phối chúng sao cho giá trị d của bộ thu phát vô tuyến là nhỏ nhất, đồng thời mọi cặp làng vẫn liên lạc được trực tiếp hoặc gián tiếp.

Ví dụ có ba làng $A(10, 10)$, $B(10, 0)$, $C(30, 0)$. Khi đó

$$|AB| = 10, \quad |BC| = 20, \quad |AC| = 10\sqrt{5} \approx 22.36.$$

Nếu không có vệ tinh hoặc chỉ có một thiết bị vệ tinh, giá trị nhỏ nhất là $d = 20$: A nối với B , B nối với C , còn A liên lạc với C qua B . Nếu có hai thiết bị vệ tinh, cấp cho B và C thì $d = 10$ là đủ. Nếu có ba thiết bị, mọi làng liên lạc trực tiếp bằng vệ tinh và $d = 0$.

Phân tích

Khi suy nghĩ thuận chiều bị bế tắc, suy nghĩ ngược chiều thường có tác dụng. Biết số vệ tinh rồi tìm khoảng cách nhỏ nhất có vẻ khó; nhưng nếu biết khoảng cách d , việc tìm số vệ tinh cần thiết rất đơn giản. Nối mọi cặp làng có thể liên lạc bằng vô tuyến, ta được một đồ thị. Số thiết bị vệ tinh cần dùng chính là số thành phần liên thông của đồ thị đó.

Bài toán trở thành: tìm d nhỏ nhất sao cho sau khi xóa mọi cạnh có trọng số lớn hơn d , số thành phần liên thông không vượt quá k .

Định lý 2. Nếu xóa mọi cạnh có trọng số lớn hơn d làm cây khung nhỏ nhất tách thành k thành phần liên thông, thì đồ thị gốc sau cùng phép xóa cũng tách thành k thành phần liên thông.

Chứng minh. Giả sử đồ thị gốc sau khi xóa các cạnh lớn hơn d có k' thành phần, với $k' \neq k$. Vì cây khung nhỏ nhất là đồ thị con của đồ thị gốc, không thể có $k' > k$. Do đó $k' < k$.

Khi đó trong một thành phần liên thông nào đó của đồ thị gốc, cây khung nhỏ nhất bị tách thành ít nhất hai phần, gọi là T_1 và T_2 . Vì chúng cùng thuộc một thành phần của đồ thị gốc sau khi xóa cạnh lớn, tồn tại $x \in T_1, y \in T_2$ sao cho $w(x, y) \leq d$. Nhưng trên đường đi từ x đến y trong cây khung nhỏ nhất phải có một cạnh (u, v) trọng số lớn hơn d ; nếu không, x và y đã không bị tách ra. Vì

$$w(x, y) \leq d < w(u, v),$$

thay (u, v) bằng (x, y) sẽ cho một cây khung có tổng trọng số nhỏ hơn, mâu thuẫn với tính nhỏ nhất. Định lý được chứng minh.

Từ định lý trên, thuật toán xây dựng rất đơn giản: đáp án là cạnh dài thứ k trong cây khung nhỏ nhất, nếu các cạnh của cây được sắp theo thứ tự giảm dần.

Chứng minh tính đúng. Trước hết, lấy d bằng cạnh dài thứ k là khả thi. Khi đó ta xóa khỏi cây khung nhỏ nhất $k - 1$ cạnh dài hơn nó, cây bị tách thành k phần. Theo Định lý 2, đồ thị gốc cũng tách thành k phần, nên k thiết bị vệ tinh đủ để nối các phần này.

Ngược lại, nếu d nhỏ hơn cạnh dài thứ k của cây khung nhỏ nhất, thì cây khung nhỏ nhất bị tách thành ít nhất $k + 1$ phần; theo Định lý 2, đồ thị gốc cũng có ít nhất $k + 1$ thành phần. Khi đó k thiết bị vệ tinh không đủ. Vậy cạnh dài thứ k của cây khung nhỏ nhất chính là giá trị nhỏ nhất của d .

Tiểu kết

Một bài toán tưởng như không có điểm bắt đầu đã được giải bằng một công cụ rất cơ bản. Trong bài này, Định lý 2 là then chốt: nó nối số thành phần liên thông của đồ thị gốc với số thành phần của cây khung nhỏ nhất sau khi xóa các cạnh lớn.

3 Kết luận

Những ứng dụng cây khung nhỏ nhất được giới thiệu trong bài chỉ là một phần rất nhỏ. Xung quanh cây khung nhỏ nhất còn nhiều định lý và ứng dụng khác, vượt xa phạm vi của một bài viết. Một số đã được phát hiện; nhiều điều thú vị hơn vẫn chờ người học sau này khai phá.

Cây khung nhỏ nhất là một trong những thuật toán đồ thị rất cơ bản. Tuy vậy, khi đào sâu, ta nhận được nhiều kết quả đáng ngạc nhiên. Không chỉ MST, nhiều thuật toán và định lý cơ sở khác cũng vậy: đằng sau vẻ ngoài đơn giản thường ẩn chứa những ý tưởng rất đáng suy ngẫm.

Về chuyện phát hiện, có bốn điều đáng nhớ. Đọc rộng tài liệu tham khảo: đứng trên vai người đi trước mới có thể nhìn xa hơn. Rèn sắc thanh kiếm trí tuệ: trí tuệ là vũ khí để vượt qua khó khăn. Xác định phương hướng lớn: chọn đúng một con đường rồi kiên định đi tiếp. Thận trọng khám phá: không bỏ qua bất kỳ dấu hiệu có giá trị nào, vì chúng có thể là điểm đột phá của vấn đề.

Lời cảm ơn

Tác giả cảm ơn Su Zhan ở trường trung học Jinling, Nanjing, vì đã giúp chứng minh Định lý 2 trong bài Arctic Network.

Tài liệu tham khảo

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, Second Edition, The MIT Press.
- Jiang Sheng, *Toward Mathematical Discovery*, Elephant Press.

A Mẫu cài đặt Prim bằng heap nhị phân

Phụ lục gốc đưa ra chương trình Pascal đầy đủ. Đoạn dưới giữ lại cấu trúc cốt lõi ở dạng giả mã để kiểm tra hơn.

```
prim_with_binary_heap(G):
    for each vertex v:
        key[v] <- infinity
        parent[v] <- NIL
        insert v into heap
    key[root] <- 0
    decrease_key(root)

    total <- 0
    while heap is not empty:
        u <- extract_min(heap)
        total <- total + key[u]
        for each edge (u, v, c):
            if v is still in heap and c < key[v]:
                key[v] <- c
                parent[v] <- u
```

```

        decrease_key(v)
    return total, parent

```

B Mẫu cài đặt Kruskal

```

kruskal(G):
    sort all edges by increasing weight
    for each vertex v:
        make_set(v)
    total <- 0
    chosen <- empty list
    for each edge (u, v, c):
        if find(u) != find(v):
            union(u, v)
            total <- total + c
            append (u, v, c) to chosen
    return total, chosen

```

C Khung cài đặt Maintain

Trong phụ lục gốc, cây khung hiện tại được lưu bằng ma trận kề. Sau khi cây đầu tiên được dựng bằng Prim, mỗi cạnh mới chỉ cần một truy vấn cạnh lớn nhất trên đường đi trong cây.

```

maintain(n, stream_edges):
    graph <- empty graph
    while graph is not connected and stream_edges is not empty:
        add next edge to graph
        if graph is connected:
            tree <- MST(graph)
            print weight(tree)
        else:
            print -1

    for each remaining edge (x, y, c):
        e <- maximum edge on path x ... y in tree
        if weight(e) > c:
            replace e by (x, y, c)
    print weight(tree)

```

D Khung cài đặt Robot

Các đỉnh của đồ thị gồm n vật cản và hai bức tường. Trọng số cạnh được tính bằng khoảng cách ℓ_∞ giữa hai vật cản, giữa vật cản với tường, hoặc giữa hai tường. Sau đó chạy Prim từ một bức tường và lấy cạnh lớn nhất trên đường đến bức tường còn lại.

```

robot(obstacles, upper_wall, lower_wall):

```

```

build complete graph on obstacles plus two wall vertices
T <- minimum spanning tree of this graph
return maximum edge on the path upper_wall ... lower_wall in T

```

E Khung cài đặt Arctic Network

```

arctic_network(points, k):
  if k < 1: k <- 1
  if k >= number_of_points: return 0
  build complete graph with Euclidean distances
  T <- minimum spanning tree
  edges <- weights of edges in T, sorted decreasing
  return edges[k]          // using one-based indexing, with edges[1] largest

```